EECS 388 Lab #6

I2C Temperature Sensor

In this lab, we will connect with a BMP180 pressure and temperature sensor. We will be using the temperature detecting capabilities in today's lab.

Task 0: Download starter code and read BMP180 Data Sheet

Task 0.0 Download starter code

Log into Canvas and go to the EECS 388 course page. Go to "Files>Labs" and under "Lab 6" click on **lab06.zip** to download the lab source code. Extract the files into a folder on your PC and open the folder in **VSCode**, same as with Lab 2-5.

Task 0.1 Read through BMP180 Data Sheet

The datasheet can be found in the **lab06/docs/BMP180.pdf.** Be sure to look through:

- Section 3 for the general operation –
  - Review figure 3 for flowchart
  - Review section 3.4 for calibration register addresses
  - Figure 4 for how to convert the raw temp ADC value to the true temperature
- Section 4 – for the sensor's register memory map
- Section 5 – General I2C information

I2C is a synchronous communication (UART is asynchronous or no clock) protocol that is used to communicate with multiple devices using a shared clock. Communication starts with the primary devices initiating a start condition, followed by a 7 bit address and 1 bit to determine if the primary device wants to read or write. After the device address is specified, the register address is specified in the next 8 bits. Then a data value is read or written. Finally, a stop condition completes the communication.

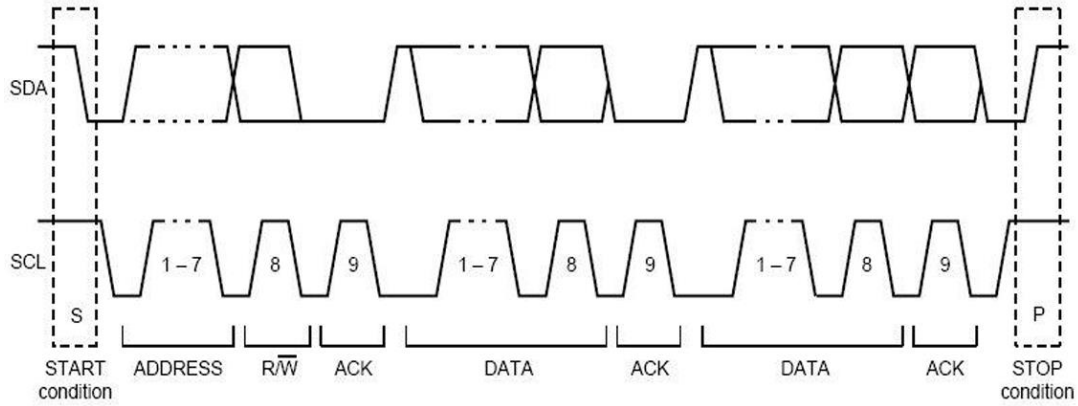

Figure 7: I^2C protocol

For this lab, we will need to read and write a few different values. The library functions to accomplish this are already setup in the code to read/write these values.

- Read:
  - id (0xD0) – The chip ID field. This will help us verify we are communicating with the chip. It should always read 0x55
  - out_lsb (0xF7)–The least significant byte of the raw temperature ADC value (ut_0 – ut_7)
  - out_msb (0xF6)–The most significant byte of the raw temperature ADC value (ut_8-ut_{15})
  - calib21_downto_calb0 (0xAA-0xBF) – Calibration values needed to convert the raw ADC value to a true temperature value in tenths of degrees Celsius.
- Write
  - ctrl_meas – Control register, sending 0x2E to this register will command a temperature read to occur

Register Name	Register Address	bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0	Reset
out_xlsb	F8h	adc_out_xlsb<7:3					0	0	0	00h
out_lsb	F7h	adc_out_msb<7:0>								00h
out_msb	F6h	adc_out_lsb<7:0								80h
ctrl_meas	F4h	oss<1:0>		sco	measurement					00h
soft	E0h	reset								00h
id	D0h	id<7:0>								55h
calib21 downto calib0	BFh *down*to AAh	calib21<7:0>*down*to calib0<7:0>								n/a

Registers:	Control	Calibration	Dat	
Type:	registers	registers	registers	Fixed

Figure 6: Memory map

The calibration values are pre-programmed in the BMP180 sensor's EEPROM. The are programmed in the factory for each individual sensor and are constant across power cycles. We will need these values to convert the raw temperature ADC value to an actual true temperature value in 0.1 degree Celsius.

Our goal for this lab is to read the raw temperature adc value "ut", and use the calibration values (AC6, AC5, MC, MD) to convert to a true temperature. See Fig 4 from the datasheet to see how these are converted using this formula:

calculate true temperature
$X1 = (UT - AC6) * AC5 / 2^{15}$
$X2 = MC * 2^{11} / (X1 + MD)$
$B5 = X1 + X2$
$T = (B5 + 8) / 2^4$

		bmp180_get_temperature	
X1 =	4743		long
X2 =	-2344		long
B5 =	2399		long
T =	**150**	*temp in 0.1°C*	long

- X1 = ((ut – AC6) * AC5) / 2^15
- X2 = (MC * 2^11) / (X1 + MD)
- B5 = X1 + X2
- T = (B5 + 8) / 2^4 (Note temp is tenths of degrees Celsius)

Task 1: BMP180 I2C programming

Task 1.0 Connect **the BMP180 to your board**

1. Connect the BMP180 Vin to the Arduino 5V supply
2. Connect the BMP180 ground to the Arduino ground (brown)
3. Connect the BMP180 SDA to the Arduino SDA pin (or A4, they are electrically connected!)
4. Connect the BMP180 SCL to the Arduino SCL pin (or A5, they are electrically connected!)

CAUTION: Review your wiring with your GTA before loading code!

Task 1.1 Verify communication with the BMP180

Use the read_register() function with the Chip ID register address to return the CHIP ID value. Confirm that I2C communication is working correctly by printing out the CHIP ID value to the serial monitor. It should read 0x55 if communication is working correctly. 0x55 is the value in the register out of reset (when the chip is powered on).

Another way to help verify communication is to check the calibration values. Those will be read from the read_calibration() function once per power cycle. Check that the printout on the serial monitor shows they are the same across power cycles.

Task 1.2 Complete the read_temperature_adc() function

The I2C function calls for reading the temperature adc value are already setup for you. The code will get the 8-bit msb, and lsb values of the raw temperature adc value 'ut'. You need to combine them into a single 16-bit ut value. This is similar to how you put together the distance for the LiDAR lab.

Task 1.3 Write the get_true_temperature() function

The true temperature can be calculated if you have the raw adc temperature 'ut' value. The equations for calculating this are in Figure 4 of the datasheet. Once you complete the function, the serial monitor should output the temperature in degrees Celsius and degrees Farenheit. You can place your hand near the sensor to see how the temperature changes.

Task 1.4 Write functions for reading the pressure and getting the true pressure value – optional 1 point bonus

You can use the datasheet to also read a raw adc value for a pressure 'up' value. The raw adc value will have to be calculated using additional calibration point information from read_calibration(). Print out the pressure value to the serial monitor and verify the value you print is accurate.

Challenge: See if you can calculate an altitude from the received pressure value. For reference, the altitude of Lawrence, KS is approximately 866 feet or ~264 meters.

Submission – 30 points (Max 32.5/30 with bonus)

Attendance: To obtain the 25% (7.5 points), you must attend the entire lab period and work toward the demo. If you are more than 30 minutes late, it will be counted as absent.

Demo: To obtain the 50% (15 points) for this lab, you must show your GTA the working temperature sensor value printing to the serial monitor. You will also need to be able to answer **some** of the following questions:

- What is the value of the R/W field in I2C for reading? 0 or 1?
- What is the value of the R/W field in I2C for writing? 0 or 1?
- Where is that value specified in the code when we start i2c communication with the BMP180 chip?
- What is the register address for the chip_id field?
- What value do we need to write to the Measure Control register to start a temperature conversion?

Code: To obtain the 25% (7.5 points) go to the "Assignments" section on Canvas and submit your modified **eecs_388_i2c.c** file to the Lab 06 submission link. Also, make sure that you have shown your completed work to your respective GTA for a demo.

Screenshots/PDF/Text files will **NOT** be allowed/graded as lab code submission. You need to submit modified **eecs_388_i2c.c** file **only**.

Bonus:
To obtain the 5% (1.5 points) you need to submit the files to Canvas by the end of your lab period. You must demo your working code on the board to your GTA before submitting the code to Canvas.

You can obtain an additional 1 point by implementing the bonus task of displaying the pressure value to the serial monitor.